

PORTADA

Análisis y Diseño de Sistemas II

Semana 11 — Modelado UML Estático: Diagrama de Clases de Diseño

Universidad de Antioquia · Ingeniería de Sistemas



ENCUADRE

Estamos en la Unidad 5

Entramos a la **Unidad 5 — Patrones de Diseño y Antipatrones**. La semana pasada (semana 10) vimos los nueve patrones **GRASP**: reglas para decidir a qué clase asignar cada responsabilidad. Ayer (martes) el equipo tomó un caso de uso simple y trazó su **diagrama de secuencia de diseño**: objetos concretos enviándose mensajes entre sí para resolver ese caso de uso.

Hoy formalizamos ese mismo diseño en un diagrama de clases — pero ya no el de dominio de la semana 4, sino su versión completa: el **diagrama de clases de diseño**.

APERTURA

**El mismo diagrama de clases
ahora listo para convertirse en código**



TRANSICIÓN

De clases conceptuales a clases de software

En la semana 4 el **diagrama de clases de dominio** mostraba solo clases conceptuales, con atributos y asociaciones — nada de cómo se implementarían en código. Esa notación mínima era intencional: en análisis todavía no existían decisiones de diseño que documentar.

Ahora sí existen: GRASP (semana 10) ya asignó responsabilidades, y ayer el diagrama de secuencia de diseño ya mostró objetos intercambiando mensajes concretos. **El diagrama de clases de diseño es donde todo eso queda registrado en una sola vista estática.**

CONCEPTO

Qué agrega el diagrama de clases de diseño

Elemento	Diagrama de clases de dominio (semana 4)	Diagrama de clases de diseño (hoy)
Atributos	Solo el nombre del concepto	Nombre + tipo de dato concreto
Visibilidad	No se declara	+ / - / # en cada atributo y operación
Operaciones	Compartimento vacío	Métodos con firma completa (parámetros y retorno)
Asociaciones	Sin dirección declarada	Navegabilidad : flecha que dice quién conoce a quién
Relaciones de uso	No existen	Dependencias (..>), distintas de la asociación estructural

Visibilidad: quién puede ver qué

Cada atributo y cada operación se marca con un símbolo de **visibilidad**, que indica desde dónde puede accederse:

Símbolo	Nombre	Significado
+	Público	Accesible desde cualquier otra clase
-	Privado	Accesible solo dentro de la propia clase
#	Protegido	Accesible desde la propia clase y sus subclases

Esta notación formaliza en UML lo que la semana 7 (**Abstracción y Encapsulamiento**) ya planteaba en principio: separar lo que una clase **expone** de lo que mantiene como **detalle interno**.

CONCEPTO · NOTACIÓN

Operaciones: los métodos, con firma completa

El tercer compartimento de la clase, vacío en el diagrama de dominio, ahora lista las **operaciones** (métodos) con su firma:

```
+ nombreDeLaOperacion(parametro1: Tipo, parametro2: Tipo): TipoDeRetorno
```

Estas operaciones no se inventan hoy: son exactamente los mismos mensajes que ayer aparecieron en el diagrama de secuencia de diseño, cuando un objeto invocaba a otro para resolver el caso de uso. Hoy simplemente se documentan del lado estático — qué clase **tiene** ese método, con qué parámetros y qué devuelve.

CONCEPTO · RELACIÓN CON LO VISTO

¿Y quién decide qué clase tiene cada método?

Que un mensaje exista en el diagrama de secuencia no dice, por sí solo, en qué clase debe vivir el método correspondiente. Esa decisión ya se tomó la semana pasada con **GRASP**:

- **Information Expert:** el método va en la clase que tiene la información necesaria para ejecutarlo.
- **Creator:** el método que crea un objeto va en la clase que lo contiene, lo agrega o tiene los datos para inicializarlo.

El diagrama de clases de diseño es, en buena medida, **el resultado visible de aplicar GRASP**: cada método aparece en la clase que GRASP indicó como responsable.

CONCEPTO · NOTACIÓN

Tipos de datos concretos en los atributos

El diagrama de dominio escribía solo el nombre del concepto (fecha , nota). El diagrama de diseño exige el **tipo de dato real** con el que se implementará:

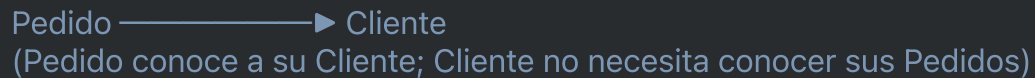
```
- fechaCreacion: LocalDate  
- cantidad: int  
- precioUnitario: BigDecimal  
# estado: EstadoPedido
```

Esta precisión es la que permite que el diagrama sirva de **punto directo hacia el código**: cualquier desarrollador puede generar la clase casi literalmente a partir de esta notación.

CONCEPTO · NOTACIÓN

Navegabilidad: quién conoce a quién

En el diagrama de dominio, una asociación era una línea sin dirección. En diseño, cada extremo puede llevar una **flecha de navegabilidad**: indica que la clase de un lado tiene una referencia (atributo) hacia la clase del otro, y por tanto puede "navegar" hasta ella.



Pedido —————▶ Cliente
(Pedido conoce a su Cliente; Cliente no necesita conocer sus Pedidos)

Una asociación puede ser unidireccional o bidireccional — y esa elección no es cosmética: define directamente cuánto **acoplamiento** (semana 7 y GRASP Low Coupling, semana 10) existe entre las dos clases.

TENSIÓN CONCEPTUAL

¿Cuánta navegabilidad es realmente necesaria?

Es tentador declarar cada asociación como **bidireccional** "por si acaso" **se necesita navegar en ambos sentidos**. Pero cada flecha de navegabilidad es una dependencia real: si `Cliente` conoce su lista de `Pedido`, cualquier cambio en `Pedido` puede afectar a `Cliente` también.

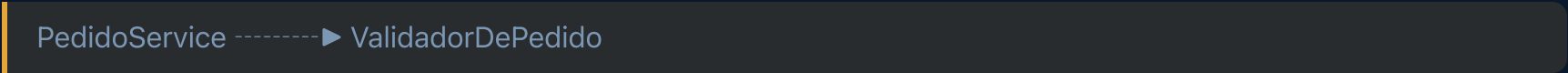
GRASP Low Coupling (semana 10) dice: declarar navegabilidad solo en el sentido que el caso de uso realmente necesita. Si nadie en el sistema necesita ir de `Cliente` a sus `Pedido`, esa dirección no se declara, aunque conceptualmente "tenga sentido".



CONCEPTO · NOTACIÓN

Dependencia: una relación de "usa", no de "conoce siempre"

Además de la asociación estructural, el diagrama de diseño introduce la **dependencia**, dibujada con una **flecha punteada** (..>):



	Asociación (línea sólida)	Dependencia (línea punteada)
Duración	Estructural: se guarda como atributo	Transitoria: solo se usa dentro de un método (parámetro o variable local)
Ejemplo	<code>Pedido</code> guarda su <code>Cliente</code> como atributo	<code>PedidoService</code> recibe un <code>ValidadorDePedido</code> solo para una llamada puntual

CONTRA EJEMPLO

Atributo público sin necesidad

Mal diseño	Problema
+ total: BigDecimal	Cualquier clase externa puede asignarle un valor arbitrario a <code>total</code> , saltándose por completo la lógica que calcula el total real del pedido

Esto rompe directamente el **encapsulamiento** (semana 7): el atributo deja de estar protegido y el objeto pierde el control sobre su propio estado consistente.

Corrección	Por qué funciona
- total: BigDecimal + calcularTotal(): BigDecimal	El atributo queda privado; el único modo de obtener el total es a través del método, que siempre lo calcula de forma consistente a partir de las líneas del pedido — sin setter público



TRANSICIÓN

Aplicando todo al caso de uso de ayer

Ayer trabajaron el caso de uso **"Registrar Pedido"**: un `Cliente` que arma un `Pedido` con una o varias `LineaDePedido`, y el sistema que lo confirma. El diagrama de secuencia de diseño mostró los objetos `PedidoController`, `PedidoService`, `Pedido` y `LineaDePedido` enviándose mensajes entre sí.

Como el sitio no dibuja diagramas UML gráficos, documentamos el mismo diagrama de clases de diseño como tabla estructurada, una por clase — igual que se hizo con la notación UML en semanas anteriores.

CONCEPTO · EJEMPLO APLICADO

Clase: Cliente

Atributos	Operaciones
- id: Long - nombre: String - correo: String	+ obtenerId(): Long + obtenerNombre(): String

Cliente es deliberadamente simple: no tiene método para crear ni calcular pedidos — esa responsabilidad no es suya según GRASP (Information Expert la ubica en otra clase, no en Cliente).

Clase: Pedido

Atributos	Operaciones
- id: Long - fechaCreacion: LocalDate - estado: EstadoPedido - lineas: List<LineaDePedido> - total: BigDecimal	+ agregarLinea(linea: LineaDePedido): void + calcularTotal(): BigDecimal + confirmar(): void

Information Expert: `Pedido` conoce sus líneas, así que es quien calcula su propio total. **Creator:** como `Pedido` contiene las `LineaDePedido` , es coherente que participe en su creación vía `agregarLinea` .

CONCEPTO · EJEMPLO APLICADO

Clase: LineaDePedido

Atributos	Operaciones
- producto: String - cantidad: int - precioUnitario: BigDecimal	+ calcularSubtotal(): BigDecimal

calcularSubtotal() vive aquí y no en Pedido por la misma razón GRASP: **LineaDePedido** es la que tiene cantidad y precioUnitario — es el **Information Expert** de ese cálculo puntual.

CONCEPTO · EJEMPLO APLICADO

Clases: PedidoController y PedidoService

Clase	Rol GRASP (semana 10)	Operaciones clave
PedidoController	Controller: puerta de entrada del caso de uso	+ registrarPedido(datos: PedidoDT0): void
PedidoService	Coordina el caso de uso completo	+ crearPedido(clienteId: Long, lineas: List<LineaDePedido>): Pedido

PedidoController no calcula nada por sí mismo — solo recibe la solicitud y delega en `PedidoService` . Esa separación es exactamente la que evita convertir al Controller en un God Object (semana 7).

CONCEPTO · EJEMPLO APLICADO

Navegabilidad, aplicada al ejemplo

Asociación	Dirección	Por qué
Pedido — Cliente	Pedido —> Cliente (unidireccional)	Pedido necesita saber de quién es; Cliente no necesita cargar la lista de sus pedidos para este caso de uso
Pedido — LineaDePedido	Pedido —> LineaDePedido (unidireccional)	Pedido gestiona sus líneas (composición); una línea no necesita navegar de vuelta a su Pedido
PedidoService — Pedido	PedidoService —> Pedido (unidireccional)	El servicio orquesta al Pedido; lo contrario aumentaría el acoplamiento sin necesidad

CONCEPTO · EJEMPLO APLICADO

Dependencia, aplicada al ejemplo

PedidoService -----► ValidadorDePedido

`PedidoService.crearPedido(...)` recibe un `ValidadorDePedido` solo dentro del método, para validar que las líneas no estén vacías antes de confirmar — no lo guarda como atributo permanente.

Por eso es dependencia y no asociación: si mañana se cambia la forma de validar, no hay que tocar la estructura interna de `PedidoService`, solo la implementación puntual que usa.

INTERACCIÓN

¿Dónde va este nuevo atributo o método?

En el chat, para cada caso decidan: **a qué clase del ejemplo pertenece**, con qué visibilidad, y si la relación involucrada es asociación o dependencia:

1. Un método que calcule el **número total de líneas** de un pedido.
2. Un atributo que guarde el **correo de contacto** alternativo de un cliente.
3. Una llamada puntual a un servicio externo de **cálculo de impuestos**, usada una sola vez dentro de `calcularTotal()` .

5 minutos · respondan en el chat

SÍNTESIS

Del dominio al diseño, en una frase

1. El diagrama de clases de diseño agrega **visibilidad** (+ - #), **operaciones con firma**, **tipos de datos concretos**, **navegabilidad** y **dependencias** al diagrama de dominio de la semana 4.
2. Los métodos que aparecen hoy son los mismos mensajes vistos ayer en el diagrama de secuencia de diseño; **GRASP (semana 10)** decide en qué clase vive cada uno.
3. La navegabilidad se diseña para minimizar acoplamiento, no "por si acaso" en ambas direcciones.
4. Una dependencia (..>) es un uso transitorio; una asociación es una relación estructural guardada como atributo.
5. Encapsular bien (semana 7) significa mantener atributos privados y exponer solo las operaciones necesarias.



CIERRE

Para el viernes

El viernes es el **Laboratorio de aplicación de GRASP**: cada equipo construye el **diagrama de clases de diseño de su propio proyecto**, aplicando exactamente lo visto hoy — visibilidad, operaciones, tipos concretos, navegabilidad y dependencias — sobre las clases y métodos que ya asignaron con GRASP la semana pasada.

